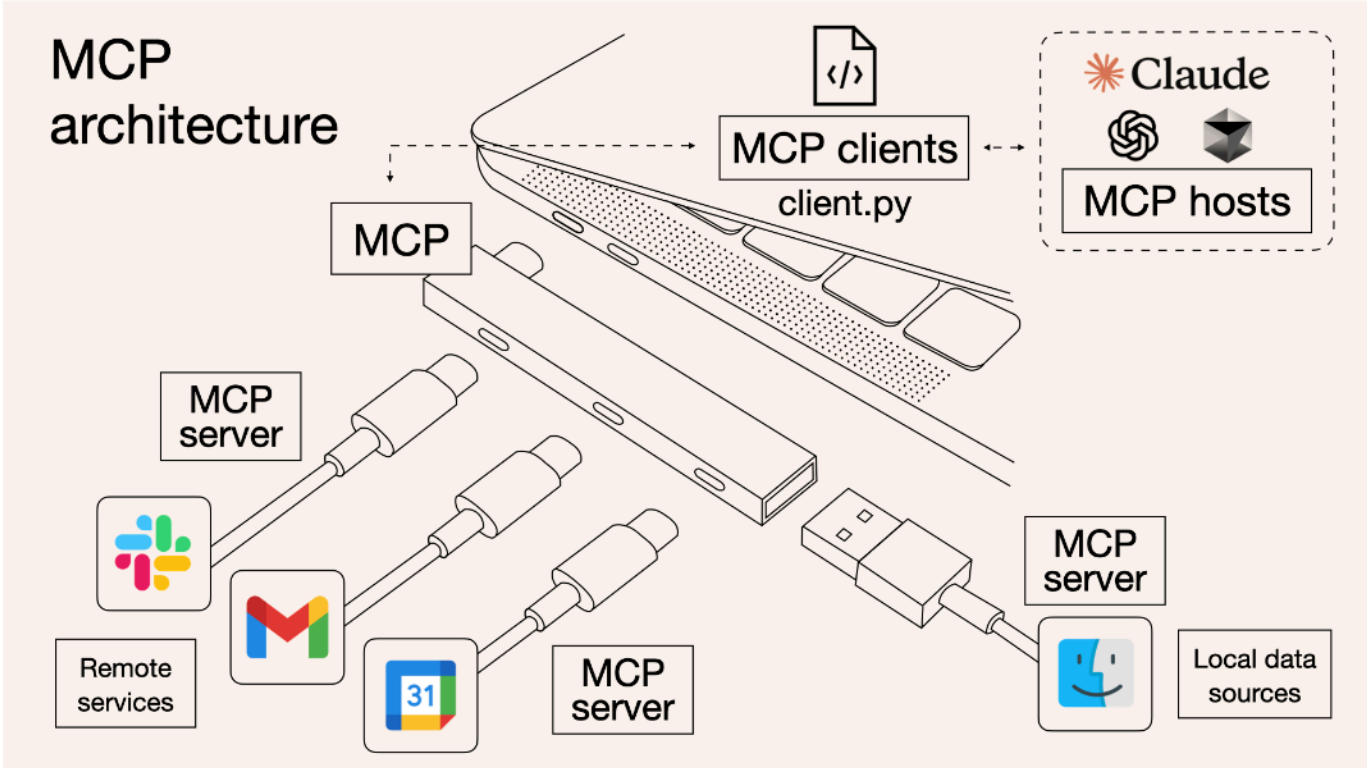
Model context protocol

A standard way to securely provide context, tools, and data to an LLM.



Problem that MCP solves?

When you use an LLM (like ChatGPT) in **real applications**, you often want it to:

- Read database schemas
- Look at files (PDFs, docs)
- Call tools or APIs
- Respect user permissions
- Use business context safely

Without a standard, **every app wires this differently**, leading to:

- Messy prompt engineering
- Security risks
- Hard-to-maintain code

MCP fixes this by standardizing how context is given to LLMs.

MCP is to LLMs what APIs are to applications i.e., a clean, secure, standardized interface for intelligence.

- **MCP (Model Context Protocol)** is a **standard way to securely provide context, tools, and data to an LLM**.
- Think of it as a **contract** between:
- Your application
- External tools/data
- The LLM

Without MCP

You talk to a human and say:

"Here's everything… schema, rules, files, permissions… please don't forget."

They forget. Or misuse it.

With MCP

You give them:

- A **structured folder**
- Clear **labels**
- Defined **tools they are allowed to use**

What does “Context” mean?

Context is **anything the model needs to answer correctly**, for example:



DATABASE TABLES & COLUMNS



USER PERMISSIONS



BUSINESS RULES



AVAILABLE TOOLS (SQL EXECUTOR, SEARCH, CALCULATOR)



DOCUMENTS / METADATA

MCP defines **how all of this is sent** to the model in a **clean, predictable way**.

Why MCP is important?

Standardization

- Same structure across apps
- No custom glue code per LLM

Security

- LLM only sees what MCP allows
- Permissions enforced outside the model

Observability

- Track what context was used
- Debug hallucinations

Model-agnostic

- Works with different LLMs
- Future-proof

Core Components of MCP



MCP SERVER



CLIENT (HOST APPLICATION)



RESOURCES



TOOLS



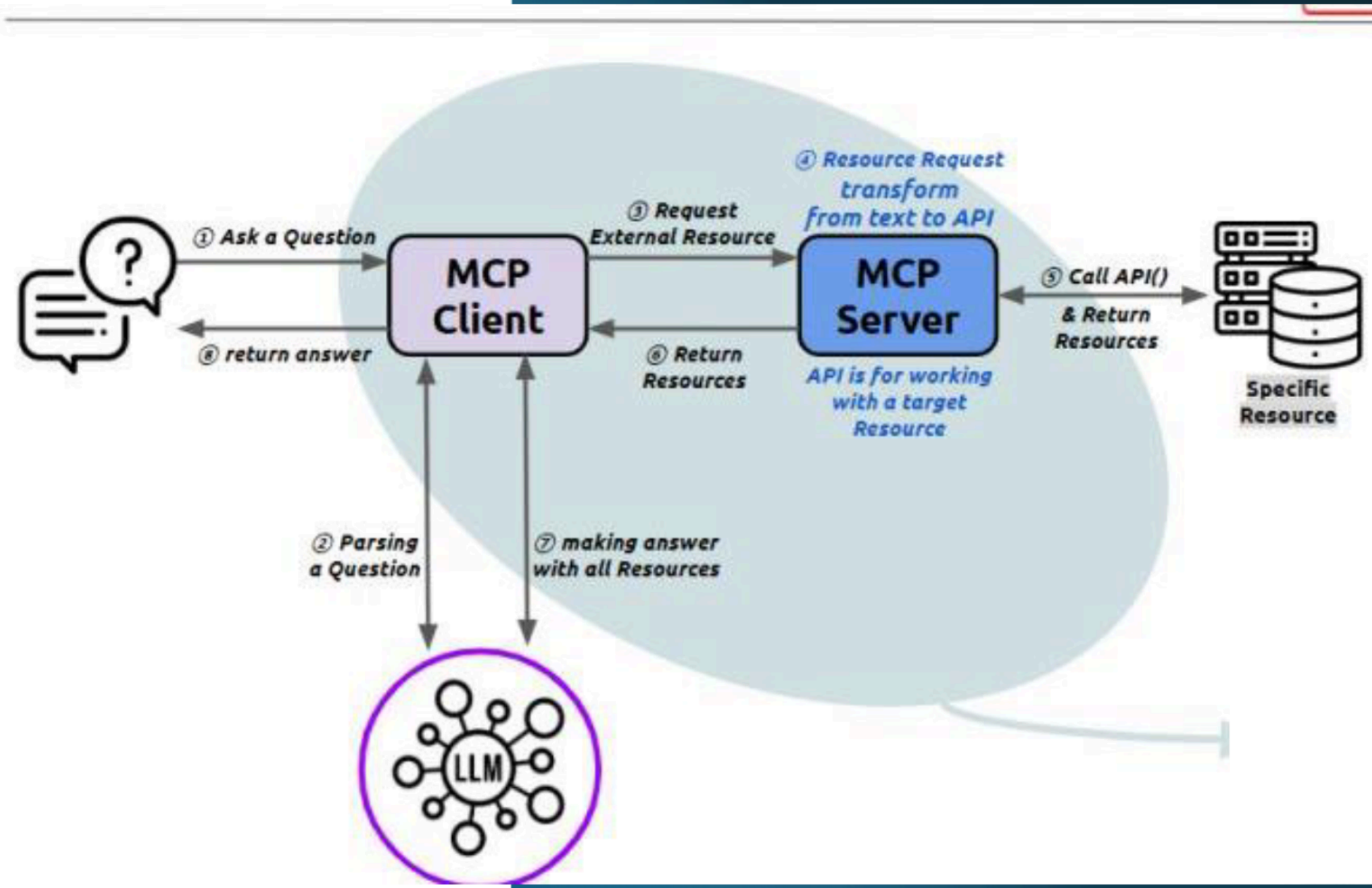
PROMPTS



CONTEXT WINDOW (STRUCTURED CONTEXT)



TRANSPORT LAYER


① Ask a Question
MCP Client
③ Request External Resource
④ Resource Request
transform from text to API
MCP Server
⑤ Call API()
& Return Resources
Specific Resource
⑧ return answer
⑥ Return Resources
API is for working with a target Resource
② Parsing a Question
⑦ making answer with all Resources
LLM

MCP Server

What it is

- A **runtime service** that exposes context to LLMs
- Acts as a **secure gateway** between your app and data/tools

What it does

- Hosts resources
- Exposes tools
- Defines prompts
- Enforces permissions

Think of it as

- “The brain-safe API layer for LLMs”

Client (Host application)

What it is

- Your UI or backend app
- Talks to the MCP server

What it does

- Web app
- Chat UI
- BI tool
- Internal platform (like Talk-to-Data)

Responsibility

- Authenticates users
- Requests context
- Sends queries to LLM

Resources

What it is

- Read-only information the model can **see but not change**

Examples

- Database schemas
- Table & column metadata
- Documents (PDF, docs)
- Configuration & policies

Tools

What it is

- **Actions** the model is allowed to perform

Examples

- run_sql(query)
- search_docs(keyword)
- call_api(payload)

Why tools matters

- Prevents hallucination
- Enables real execution
- Keeps logic outside the LLM

Prompts

What it is

- Predefined **instruction templates**

Types

- System prompts
- Business rules
- Style & formatting rules

Example

- "You are a data analyst. Always return SQL first, then explanation."

Context Window (Structured Context)

What it is

- The **assembled package** of:
  - Resources
  - Tool definitions
  - Prompts

What it does

- Context is structured, not dumped
- Deterministic and auditable

Transport Layer

What it is

- How messages move between:
  - Client ↔ MCP Server ↔ LLM

Examples:

- HTTP
- stdio
- WebSockets

THANKS!